

BRICK BREAKER

LAB IMPLEMENTATION REPORT

Pygame • Iterative LLM Debugging & Pair Programming

NAME	SRN	SECTION
VRUSHANT K P	PES1UG24CS542	I

01 • LLM / CHAT HISTORY

Complete ChatGPT conversation used during the iterative analysis, debugging, implementation, and testing process:

<https://chatgpt.com/share/6abd3a54-3d20-83ee-b89c-d217dc0fabea>

02 • PROJECT OVERVIEW

The project is a Pygame-based Brick Breaker (Breakout-style) game. The starter version provided basic paddle, ball, brick, score, and life functionality, but required improvements to collision handling, game completion states, replay functionality, difficulty selection, and sound feedback. The work was completed iteratively with an LLM used as a debugging and pair-programming partner, followed by manual testing and review.

03 • TASK 1 — REFINE COLLISION DETECTION

Problem: The original collision logic reversed only the ball's vertical velocity when a brick or paddle was hit, so side and corner collisions could produce incorrect bounce directions.

Fix: Collision handling was improved using previous/current ball position and collision geometry to determine the side actually contacted. Horizontal velocity is reversed for left/right impacts, while vertical velocity is reversed for top/bottom impacts. Collision separation was also handled to reduce overlap and repeated collision problems.

Testing: Normal, side, corner, adjacent-brick, paddle-edge, high-speed, and stuck/overlap cases were reviewed and tested during gameplay.

04 • TASK 2 — GAME OVER / WIN SCREEN

Problem: The original game detected win/loss states but reported them through the console rather than displaying a proper in-game result.

Fix: A graphical end screen was added for both winning and losing. It displays the result and final score and waits for player input instead of immediately terminating the game.

05 • TASK 3 — REPLAY AND DIFFICULTY SELECTION

Problem: The starter project had no replay flow or selectable difficulty levels.

Fix: After a game ends, the player can choose Easy, Medium, Hard, or Exit. Difficulty changes ball speed and paddle size. A fresh game resets score, lives, bricks, ball position/velocity, paddle state, and other relevant game state.

Testing: Replay, state reset, difficulty changes, and exiting the application were tested.

06 • TASK 4 — SOUND FEEDBACK

Problem: The starter project contained no audio feedback.

Fix: Basic Pygame sound effects were added for paddle hits, wall hits, brick destruction, and win/game-over events. Sound assets were organized separately, and missing/unavailable audio is handled without crashing the game.

Testing: The completed sound behavior was tested during gameplay and confirmed to work correctly.

07 • FINAL RESULT

All four requested tasks were implemented while preserving the existing project structure.

The final game provides accurate collision responses, functional lives and scoring, graphical win/loss feedback, replay with selectable difficulty, and sound effects. The completed project was manually tested after implementation and the required functionality was confirmed to work.

ANALYZE



LLM SUGGESTION



CRITICAL REVIEW



IMPLEMENT



TEST & REFINE

Implementation approach: Analyze → LLM suggestion → critical review → implementation → manual testing → refinement.